JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LinkedTransferQueue<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>
      java.util.concurrent.LinkedTransferQueue<E>

Type Parameters:

E - the type of elements held in this queue

All Implemented Interfaces:

Serializable, Iterable<E>, Collection<E>, BlockingQueue<E>, TransferQueue<E>, Queue<E>

```
public class LinkedTransferQueue<E>
extends AbstractQueue<E>
implements TransferQueue<E>, Serializable
```

An unbounded TransferQueue based on linked nodes. This queue orders elements FIFO (first-in-first-out) with respect to any given producer. The *head* of the queue is that element that has been on the queue the longest time for some producer. The *tail* of the queue is that element that has been on the queue the shortest time for some producer.

Beware that, unlike in most collections, the `size` method is *NOT* a constant-time operation. Because of the asynchronous nature of these queues, determining the current number of elements requires a traversal of the elements, and so may report inaccurate results if this collection is modified during traversal.

Bulk operations that add, remove, or examine multiple elements, such as AbstractQueue.addAll(Collection), removeIf(Predicate) or forEach(Consumer), are *not* guaranteed to be performed atomically. For example, a `forEach` traversal concurrent with an `addAll` operation might observe only some of the added elements.

This class and its iterator implement all of the *optional* methods of the Collection and Iterator interfaces.

Memory consistency effects: As with other concurrent collections, actions in a thread prior to placing an object into a `LinkedTransferQueue` *happen-before* actions subsequent to the access or removal of that element from the `LinkedTransferQueue` in another thread.

This class is a member of the Java Collections Framework.

Since:

1.7

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`LinkedTransferQueue()`	Creates an initially empty `LinkedTransferQueue`.
`LinkedTransferQueue(Collection<? extends E> c)`	Creates a `LinkedTransferQueue` initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`boolean`	`add(E e)`	Inserts the specified element at the tail of this queue.
`boolean`	`contains(Object o)`	Returns `true` if this queue contains the specified element.
`int`	`drainTo(Collection<? super E> c)`	Removes all available elements from this queue and adds them to the given collection.
`int`	`drainTo(Collection<? super E> c, int maxElements)`	Removes at most the given number of available elements from this queue and adds them to the given collection.
`void`	`forEach(Consumer<? super E> action)`	Performs the given action for each element of the `Iterable` until all

		elements have been processed or the action throws an exception.
int	**getWaitingConsumerCount**(	Returns an estimate of the number of consumers waiting to receive elements via `BlockingQueue.take()` or timed `poll`.
boolean	**hasWaitingConsumer**()	Returns `true` if there is at least one consumer waiting to receive an element via `BlockingQueue.take()` or timed `poll`.
boolean	**isEmpty**()	Returns `true` if this queue contains no elements.
`Iterator<E>`	**iterator**()	Returns an iterator over the elements in this queue in proper sequence.
boolean	**offer**(E e)	Inserts the specified element at the tail of this queue.
boolean	**offer**(E e, long timeout, TimeUnit unit)	Inserts the specified element at the tail of this queue.
E	**peek**()	Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
E	**poll**()	Retrieves and removes the head of this queue,

		or returns `null` if this queue is empty.
`E`	**`poll`**`(long timeout, TimeUnit unit)`	Retrieves and removes the head of this queue, waiting up to the specified wait time if necessary for an element to become available.
`void`	**`put`**`(E e)`	Inserts the specified element at the tail of this queue.
`int`	**`remainingCapacity`**`()`	Always returns `Integer.MAX_VALUE` because a `LinkedTransferQueue` is not capacity constrained.
`boolean`	**`remove`**`(Object o)`	Removes a single instance of the specified element from this queue, if it is present.
`boolean`	**`removeAll`**`(Collection<?> c)`	Removes all of this collection's elements that are also contained in the specified collection (optional operation).
`boolean`	**`removeIf`**`(Predicate<? super E> filter)`	Removes all of the elements of this collection that satisfy the given predicate (optional operation).
`boolean`	**`retainAll`**`(Collection<?> c)`	Retains only the elements in this collection that are contained in the specified collection (optional operation).

int	**size**()	Returns the number of elements in this queue.
Spliterator<E>	**spliterator**()	Returns a Spliterator over the elements in this queue.
E	**take**()	Retrieves and removes the head of this queue, waiting if necessary until an element becomes available.
Object[]	**toArray**()	Returns an array containing all of the elements in this queue, in proper sequence.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this queue, in proper sequence; the runtime type of the returned array is that of the specified array.
void	**transfer**(E e)	Transfers the element to a consumer, waiting if necessary to do so.
boolean	**tryTransfer**(E e)	Transfers the element to a waiting consumer immediately, if possible.
boolean	**tryTransfer**(E e, long timeout, TimeUnit unit)	Transfers the element to a consumer if it is possible to do so before the timeout elapses.

Methods declared in class AbstractQueue

addAll, clear, element, remove

Methods declared in class AbstractCollection

containsAll, toString

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods declared in interface Collection

equals, hashCode, parallelStream, stream, toArray

Constructor Details

LinkedTransferQueue

```
public LinkedTransferQueue()
```

Creates an initially empty `LinkedTransferQueue`.

LinkedTransferQueue

```
public LinkedTransferQueue(Collection<? extends E> c)
```

Creates a `LinkedTransferQueue` initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Parameters:

`c` - the collection of elements to initially contain

Throws:

`NullPointerException` - if the specified collection or any of its elements are null

Method Details

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this queue, in proper sequence.

The returned array will be "safe" in that no references to it are maintained by this queue. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Returns:

an array containing all of the elements in this queue

toArray

`public <T> T[] toArray(T[] a)`

Returns an array containing all of the elements in this queue, in proper sequence; the runtime type of the returned array is that of the specified array. If the queue fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this queue.

If this queue fits in the specified array with room to spare (i.e., the array has more elements than this queue), the element in the array immediately following the end of the queue is set to `null`.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a queue known to contain only strings. The following code can be used to dump the queue into a newly allocated array of `String`:

```
String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Type Parameters:

`T` - the component type of the array to contain the collection

Parameters:

a - the array into which the elements of the queue are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose

Returns:

an array containing all of the elements in this queue

Throws:

`ArrayStoreException` - if the runtime type of the specified array is not a supertype of the runtime type of every element in this queue

`NullPointerException` - if the specified array is null

spliterator

```
public Spliterator<E> spliterator()
```

Returns a `Spliterator` over the elements in this queue.

The returned spliterator is *weakly consistent*.

The `Spliterator` reports `Spliterator.CONCURRENT`, `Spliterator.ORDERED`, and `Spliterator.NONNULL`.

Specified by:

`spliterator` in interface `Collection<E>`

Specified by:

`spliterator` in interface `Iterable<E>`

Implementation Note:

The `Spliterator` implements `trySplit` to permit limited parallelism.

Returns:

a `Spliterator` over the elements in this queue

Since:

1.8

put

```
public void put(E e)
```

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never block.

Specified by:

`put` in interface `BlockingQueue<E>`

Parameters:

e - the element to add

Throws:

`NullPointerException` - if the specified element is null

offer

```
public boolean offer(E e,
                     long timeout,
                     TimeUnit unit)
```

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never block or return `false`.

Specified by:

`offer` in interface `BlockingQueue<E>`

Parameters:

`e` - the element to add

`timeout` - how long to wait before giving up, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `timeout` parameter

Returns:

`true` (as specified by `BlockingQueue.offer`)

Throws:

`NullPointerException` - if the specified element is null

offer

```
public boolean offer(E e)
```

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never return `false`.

Specified by:

`offer` in interface `BlockingQueue<E>`

Specified by:

`offer` in interface `Queue<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Queue.offer(E)`)

Throws:

`NullPointerException` - if the specified element is null

add

`public boolean add(E e)`

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never throw `IllegalStateException` or return `false`.

Specified by:

`add` in interface `BlockingQueue<E>`

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `Queue<E>`

Overrides:

`add` in class `AbstractQueue<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`NullPointerException` - if the specified element is null

tryTransfer

`public boolean tryTransfer(E e)`

Transfers the element to a waiting consumer immediately, if possible.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `take()` or timed `poll`), otherwise returning `false` without enqueuing the element.

Specified by:

`tryTransfer` in interface `TransferQueue<E>`

Parameters:

`e` - the element to transfer

Returns:

`true` if the element was transferred, else `false`

Throws:

`NullPointerException` - if the specified element is null

transfer

```
public void transfer(E e)
              throws InterruptedException
```

Transfers the element to a consumer, waiting if necessary to do so.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `take()` or timed `poll`), else inserts the specified element at the tail of this queue and waits until the element is received by a consumer.

Specified by:

`transfer` in interface `TransferQueue<E>`

Parameters:

`e` - the element to transfer

Throws:

`NullPointerException` - if the specified element is null

`InterruptedException` - if interrupted while waiting, in which case the element is not left enqueued

tryTransfer

```
public boolean tryTransfer(E e,
                           long timeout,
                           TimeUnit unit)
                    throws InterruptedException
```

Transfers the element to a consumer if it is possible to do so before the timeout elapses.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `take()` or timed `poll`), else inserts the specified element at the tail of this queue and waits until the element is received by a consumer, returning `false` if the specified wait time elapses before the element can be transferred.

Specified by:

`tryTransfer` in interface `TransferQueue<E>`

Parameters:

`e` - the element to transfer

`timeout` - how long to wait before giving up, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `timeout` parameter

Returns:

`true` if successful, or `false` if the specified waiting time elapses before completion, in which case the element is not left enqueued

Throws:

`NullPointerException` - if the specified element is null

`InterruptedException` - if interrupted while waiting, in which case the element is not left enqueued

take

```
public E take()
       throws InterruptedException
```

Description copied from interface: `BlockingQueue`

Retrieves and removes the head of this queue, waiting if necessary until an element becomes available.

Specified by:

`take` in interface `BlockingQueue<E>`

Returns:

the head of this queue

Throws:

`InterruptedException` - if interrupted while waiting

poll

```
public E poll(long timeout,
              TimeUnit unit)
       throws InterruptedException
```

Description copied from interface: `BlockingQueue`

Retrieves and removes the head of this queue, waiting up to the specified wait time if necessary for an element to become available.

Specified by:

`poll` in interface `BlockingQueue<E>`

Parameters:

`timeout` - how long to wait before giving up, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `timeout` parameter

Returns:

the head of this queue, or `null` if the specified waiting time elapses before an element is available

Throws:

InterruptedException - if interrupted while waiting

poll

```
public E poll()
```

Description copied from interface: Queue

Retrieves and removes the head of this queue, or returns `null` if this queue is empty.

Specified by:

poll in interface Queue<E>

Returns:

the head of this queue, or `null` if this queue is empty

drainTo

```
public int drainTo(Collection<? super E> c)
```

Description copied from interface: BlockingQueue

Removes all available elements from this queue and adds them to the given collection. This operation may be more efficient than repeatedly polling this queue. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

drainTo in interface BlockingQueue<E>

Parameters:

c - the collection to transfer elements into

Returns:

the number of elements transferred

Throws:

NullPointerException - if the specified collection is null

IllegalArgumentException - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

drainTo

```
public int drainTo(Collection<? super E> c,
                   int maxElements)
```

Description copied from interface: `BlockingQueue`

Removes at most the given number of available elements from this queue and adds them to the given collection. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`drainTo` in interface `BlockingQueue<E>`

Parameters:

`c` - the collection to transfer elements into

`maxElements` - the maximum number of elements to transfer

Returns:

the number of elements transferred

Throws:

`NullPointerException` - if the specified collection is null

`IllegalArgumentException` - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this queue in proper sequence. The elements will be returned in order from first (head) to last (tail).

The returned iterator is *weakly consistent*.

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an iterator over the elements in this queue in proper sequence

peek

```
public E peek()
```

Description copied from interface: Queue

Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.

Specified by:

`peek` in interface `Queue<E>`

Returns:

the head of this queue, or `null` if this queue is empty

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this queue contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

`true` if this queue contains no elements

hasWaitingConsumer

```
public boolean hasWaitingConsumer()
```

Description copied from interface: TransferQueue

Returns `true` if there is at least one consumer waiting to receive an element via `BlockingQueue.take()` or timed `poll`. The return value represents a momentary state of affairs.

Specified by:

`hasWaitingConsumer` in interface `TransferQueue<E>`

Returns:

`true` if there is at least one waiting consumer

size

```
public int size()
```

Returns the number of elements in this queue. If this queue contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Beware that, unlike in most collections, this method is *NOT* a constant-time operation. Because of the asynchronous nature of these queues, determining the current number of elements requires an O(n) traversal.

Specified by:

`size` in interface `Collection<E>`

Returns:

the number of elements in this queue

getWaitingConsumerCount

```
public int getWaitingConsumerCount()
```

Description copied from interface: `TransferQueue`

Returns an estimate of the number of consumers waiting to receive elements via `BlockingQueue.take()` or timed `poll`. The return value is an approximation of a momentary state of affairs, that may be inaccurate if consumers have completed or given up waiting. The value may be useful for monitoring and heuristics, but not for synchronization control. Implementations of this method are likely to be noticeably slower than those for `TransferQueue.hasWaitingConsumer()`.

Specified by:

`getWaitingConsumerCount` in interface `TransferQueue<E>`

Returns:

the number of consumers waiting to receive elements

remove

```
public boolean remove(Object o)
```

Removes a single instance of the specified element from this queue, if it is present. More formally, removes an element `e` such that `o.equals(e)`, if this queue contains one or more such elements. Returns `true` if this queue contained the specified element (or equivalently, if this queue changed as a result of the call).

Specified by:

`remove` in interface `BlockingQueue<E>`

Specified by:

`remove` in interface `Collection<E>`

Overrides:

remove in class AbstractCollection<E>

Parameters:

o - element to be removed from this queue, if present

Returns:

true if this queue changed as a result of the call

contains

```
public boolean contains(Object o)
```

Returns true if this queue contains the specified element. More formally, returns true if and only if this queue contains at least one element e such that o.equals(e).

Specified by:

contains in interface BlockingQueue<E>

Specified by:

contains in interface Collection<E>

Overrides:

contains in class AbstractCollection<E>

Parameters:

o - object to be checked for containment in this queue

Returns:

true if this queue contains the specified element

remainingCapacity

```
public int remainingCapacity()
```

Always returns Integer.MAX_VALUE because a LinkedTransferQueue is not capacity constrained.

Specified by:

remainingCapacity in interface BlockingQueue<E>

Returns:

Integer.MAX_VALUE (as specified by BlockingQueue.remainingCapacity())

removeIf

```
public boolean removeIf(Predicate<? super E> filter)
```

Description copied from interface: Collection

Removes all of the elements of this collection that satisfy the given predicate (optional operation). Errors or runtime exceptions thrown during iteration or by the predicate are relayed to the caller.

Specified by:

`removeIf` in interface `Collection<E>`

Parameters:

`filter` - a predicate which returns `true` for elements to be removed

Returns:

`true` if any elements were removed

Throws:

`NullPointerException` - if the specified filter is null

removeAll

```
public boolean removeAll(Collection<?> c)
```

Description copied from class: `AbstractCollection`

Removes all of this collection's elements that are also contained in the specified collection (optional operation). After this call returns, this collection will contain no elements in common with the specified collection.

Specified by:

`removeAll` in interface `Collection<E>`

Overrides:

`removeAll` in class `AbstractCollection<E>`

Parameters:

`c` - collection containing elements to be removed from this collection

Returns:

`true` if this collection changed as a result of the call

Throws:

`NullPointerException` - if this collection contains one or more null elements and the specified collection does not support null elements (optional) or if the specified collection is null

See Also:

`AbstractCollection.remove(Object)`, `AbstractCollection.contains(Object)`

retainAll

```
public boolean retainAll(Collection<?> c)
```

Description copied from class: `AbstractCollection`

Retains only the elements in this collection that are contained in the specified collection (optional operation). In other words, removes from this collection all of its elements that are not contained in the specified collection.

Specified by:

`retainAll` in interface `Collection<E>`

Overrides:

`retainAll` in class `AbstractCollection<E>`

Parameters:

`c` - collection containing elements to be retained in this collection

Returns:

`true` if this collection changed as a result of the call

Throws:

`NullPointerException` - if this collection contains one or more null elements and the specified collection does not permit null elements (optional) or if the specified collection is null

See Also:

`AbstractCollection.remove(Object)`, `AbstractCollection.contains(Object)`

forEach

```
public void forEach(Consumer<? super E> action)
```

Description copied from interface: `Iterable`

Performs the given action for each element of the `Iterable` until all elements have been processed or the action throws an exception. Actions are performed in the order of iteration, if that order is specified. Exceptions thrown by the action are relayed to the caller.

The behavior of this method is unspecified if the action performs side-effects that modify the underlying source of elements, unless an overriding class has specified a concurrent modification policy.

Specified by:

`forEach` in interface `Iterable<E>`

Parameters:

`action` - The action to be performed for each element

Throws:

`NullPointerException` - if the specified action is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.